

Rajalakshmi Engineering College

Name: M.S Nanthana shree
Email: 241901064@rajalakshmi.edu.in
Roll no: 241901064
Phone: 9360753427
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 6_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Teena's retail store has implemented a Loyalty Points System to reward customers based on their spending. The program calculates and displays the loyalty points based on whether the customer is a regular or a premium customer.

For regular customers (class Customer), the loyalty points are calculated as:

$\text{Loyalty points} = \text{amount spent} / 10$

For premium customers (class PremiumCustomer, which inherits from Customer), the loyalty points are calculated as:

$\text{Loyalty points} = 2 * (\text{amount spent} / 10)$

The program should use method overriding for premium customers to

calculate their loyalty points. The method that needs to be overridden is calculateLoyaltyPoints in the Customer class.

Input Format

The first line of input consists of an integer representing the amount spent by the customer.

The second line consists of a string representing the premium customer status:

- "yes" if the customer is a premium customer.
- "no" if the customer is not a premium customer.

Output Format

The output should display the loyalty points earned based on the amount spent and the customer type.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

yes

Output: 10

Answer

```
import java.util.Scanner;
import java.util.Scanner;

// Base class for Customer
class Customer {
    protected int amountSpent;

    public Customer(int amountSpent) {
        this.amountSpent = amountSpent;
    }

    // Method to calculate loyalty points
    public int calculateLoyaltyPoints() {
```

```

        return amountSpent / 10;
    }
}

// PremiumCustomer class inherits from Customer
class PremiumCustomer extends Customer {

    public PremiumCustomer(int amountSpent) {
        super(amountSpent);
    }

    @Override
    public int calculateLoyaltyPoints() {
        return 2 * (amountSpent / 10);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int amountSpent = scanner.nextInt();

        String isPremium = scanner.next().toLowerCase();

        Customer customer;

        if (isPremium.equals("yes")) {
            customer = new PremiumCustomer();
        } else {
            customer = new Customer();
        }

        int loyaltyPoints = customer.calculateLoyaltyPoints(amountSpent);

        System.out.println(loyaltyPoints);
    }
}

```

Status : Wrong

Marks : 0/10

2. Problem Statement

A painter needs to determine the cost to paint different shapes based on their surface area. The program should be designed to handle the area of a sphere and calculate the total painting cost using the following formulas:

Area of sphere: $\text{Area} = 4 * \pi * r^2$ where $\pi = 3.14$
Total painting cost: $\text{Cost} = \text{cost per square meter} * \text{area of sphere}$

The program will consist of three classes:

Shape class: This class should set the shape type and radius.

Area class: This class should extend Shape to calculate the area.

Cost class: This class should extend Area to calculate the total painting cost.

Input Format

The input consists of a string representing the shape type, a double value representing the radius, and another double value representing the cost per square meter on each line.

Output Format

For a valid shape type of "Sphere":

- The first line prints: "Area of Sphere is: <calculated_area>" rounded to two decimal places.
- The second line prints: "Cost to paint the shape is: <total_painting_cost>" rounded to two decimal places.

For any other shape types, print: "Invalid type".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Sphere

3.4

5.8

Output: Area of Sphere is: 145.19

Cost to paint the shape is: 842.12

Answer

```
import java.util.Scanner;
import java.util.Scanner;

// Base class Shape
class Shape {
    protected String type;
    protected double radius;

    public Shape(String type, double radius) {
        this.type = type;
        this.radius = radius;
    }
}

// Area class extending Shape to calculate area
class Area extends Shape {
    protected double area;

    public Area(String type, double radius) {
        super(type, radius);
    }

    public void calculateArea() {
        if ("Sphere".equals(type)) {
            area = 4 * 3.14 * radius * radius;
        } else {
            area = -1; // invalid type
        }
    }

    public double getArea() {
        return area;
    }
}

// Cost class extending Area to calculate total painting cost
class Cost extends Area {
    private double costPerSqM;
    private double totalCost;
```

```

public Cost(String type, double radius, double costPerSqM) {
    super(type, radius);
    this.costPerSqM = costPerSqM;
}

public void calculateTotalCost() {
    calculateArea();
    if (area != -1) {
        totalCost = area * costPerSqM;
    }
}

public void display() {
    if (area == -1) {
        System.out.println("Invalid type");
    } else {
        System.out.printf("Area of Sphere is: %.2f%n", area);
        System.out.printf("Cost to paint the shape is: %.2f%n", totalCost);
    }
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String s = scanner.next();
        Cost shape = new Cost();
        shape.setShape(s, scanner);
        double costToPaint = scanner.nextDouble();
        shape.calculateArea();
        shape.setCost(costToPaint);
        shape.calculateCost();
    }
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

Mary is managing a business and wants to analyze its profitability. She

operates both a regular business model and a seasonal business model. To assess profitability, she uses a program that calculates and compares the profit margins for both models based on revenue and cost.

The program defines:

BusinessUtility class with a method calculateMargin(double revenue, double cost). SeasonalBusinessUtility (inherits from BusinessUtility) and overrides calculateMargin(double revenue, double cost), adding a seasonal adjustment of 10% to the base margin. ProfitabilityChecker class with a method checkProfitability(double regularMargin), which prints "Business is profitable." if the regular margin is 10% or more, otherwise prints "Business is not profitable."

Mary inputs revenue and cost, and the program compute and display the regular and seasonal margins using:

$$\text{Margin} = ((\text{Revenue} - \text{Cost}) / \text{Revenue}) \times 100$$
$$\text{Seasonal Margin} = \text{Margin} + 10$$

Input Format

The first line of input consists of a double value r , representing the revenue.

The second line consists of a double value c , representing the cost.

Output Format

The first line prints a double value, representing the regular profit margin, rounded to two decimal places, in the format: "Regular Margin: X. XX%", where X.XX denotes the calculated regular margin.

The second line prints a double value, representing the seasonal profit margin, rounded to two decimal places, in the format: "Seasonal Margin: X. XX%", where X.XX denotes the calculated seasonal margin.

The third line prints a string, indicating whether the business is profitable or not profitable, based on the regular margin.

If the regular margin is less than 10, print "Business is not profitable.". If it is 10 or greater, print "Business is profitable."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1000.0

800.0

Output: Regular Margin: 20.00%

Seasonal Margin: 30.00%

Business is profitable.

Answer

```
import java.util.Scanner;
```

```
// You are using Java
```

```
import java.util.Scanner;
```

```
// Base class for business utility
```

```
class BusinessUtility {
```

```
    public double calculateMargin(double revenue, double cost) {  
        return ((revenue - cost) / revenue) * 100;  
    }
```

```
}
```

```
// Subclass for seasonal business, adds 10% seasonal adjustment
```

```
class SeasonalBusinessUtility extends BusinessUtility {
```

```
    @Override
```

```
    public double calculateMargin(double revenue, double cost) {  
        double baseMargin = super.calculateMargin(revenue, cost);  
        return baseMargin + 10;  
    }
```

```
}
```

```
// Class to check profitability
```

```
class ProfitabilityChecker {
```

```
    public void checkProfitability(double regularMargin) {  
        if (regularMargin >= 10) {  
            System.out.println("Business is profitable.");  
        } else {  
            System.out.println("Business is not profitable.");  
        }  
    }
```



```

    }
}
class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double revenue = scanner.nextDouble();
        double cost = scanner.nextDouble();
        BusinessUtility business = new BusinessUtility();
        SeasonalBusinessUtility seasonalBusiness = new
SeasonalBusinessUtility();
        double regularMargin = business.calculateMargin(revenue, cost);
        double seasonalMargin = seasonalBusiness.calculateMargin(revenue,
cost);

        System.out.printf("Regular Margin: %.2f%%\n", regularMargin);
        System.out.printf("Seasonal Margin: %.2f%%\n", seasonalMargin);

        ProfitabilityChecker checker = new ProfitabilityChecker();
        checker.checkProfitability(regularMargin);
        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

A bank provides two types of deposit schemes: Fixed Deposits (FD) and Recurring Deposits (RD). Customers want to calculate the interest they can earn based on their selected scheme.

Develop a Java program using inheritance to compute the interest for FD and RD. The program should include:

A base class Account with attributes accountHolder and principalAmount, along with a method for interest calculation. A subclass FixedDeposit that calculates interest for FD. A subclass RecurringDeposit that calculates interest for RD.

Formulas Used:

Interest for FD: $(\text{principal amount} * \text{duration in years} * \text{rate of interest}) / 100$

Interest for RD: $(\text{maturity amount} * \text{duration in months} * \text{rate of interest}) / (12 * 100)$, where maturity amount = monthly deposit * duration in months.

Input Format

The first line of input consists of the choice (1 for FD, 2 for RD).

If the choice is 1, the following lines consist of account holder (string), principal amount (double), duration in years (int), and rate of interest (double).

If the choice is 2, the following lines consist of account holder (string), monthly deposit (int), duration in months (int), and rate of interest (double).

Output Format

The output prints the calculated interest with one decimal place in the following format.

For choice 1: "Interest for FD: <calculated interest >"

For choice 2: "Interest for FD: <calculated interest >"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Alice

50000.56

5

6.5

Output: Interest for FD: 16250.2

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
// Base class Account
```

```
class Account {
```

```
protected String accountHolder;

public Account(String accountHolder) {
    this.accountHolder = accountHolder;
}

// Placeholder method for interest calculation
public double calculateInterest() {
    return 0.0;
}
}

// Subclass for Fixed Deposit
class FixedDeposit extends Account {
    private double principal;
    private int durationYears;
    private double rate;

    public FixedDeposit(String accountHolder, double principal, int durationYears,
double rate) {
        super(accountHolder);
        this.principal = principal;
        this.durationYears = durationYears;
        this.rate = rate;
    }

    @Override
    public double calculateInterest() {
        return (principal * durationYears * rate) / 100.0;
    }
}

// Subclass for Recurring Deposit
class RecurringDeposit extends Account {
    private int monthlyDeposit;
    private int durationMonths;
    private double rate;

    public RecurringDeposit(String accountHolder, int monthlyDeposit, int
durationMonths, double rate) {
        super(accountHolder);
        this.monthlyDeposit = monthlyDeposit;
    }
}
```

```

        this.durationMonths = durationMonths;
        this.rate = rate;
    }

    @Override
    public double calculateInterest() {
        double maturityAmount = monthlyDeposit * durationMonths;
        return (maturityAmount * durationMonths * rate) / (12 * 100.0);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int choice = sc.nextInt();

        switch (choice) {
            case 1:
                sc.nextLine();
                String fdName = sc.nextLine();
                double fdPrincipal = sc.nextDouble();
                int fdDuration = sc.nextInt();
                double fdRate = sc.nextDouble();

                FixedDeposit fd = new FixedDeposit(fdName, fdPrincipal, fdDuration,
fdRate);
                System.out.printf("Interest for FD: %.1f", fd.calculateInterest());
                break;

            case 2:
                sc.nextLine();
                String rdName = sc.nextLine();
                int rdDeposit = sc.nextInt();
                int rdDuration = sc.nextInt();
                double rdRate = sc.nextDouble();

                RecurringDeposit rd = new RecurringDeposit(rdName, rdDeposit,
rdDuration, rdRate);
                System.out.printf("Interest for RD: %.1f", rd.calculateInterest());
                break;
        }
    }
}

```

```
default:  
    System.out.println("Invalid Choice");  
    }  
    }  
}
```

Status : Correct

Marks : 10/10